
DEEP RESEARCH · FROM CLAUDE-DESIGNED PROTOTYPE TO A LIVE PRODUCT

LEXFIT: turning the design into a real, data-driven app.

This is the master plan for taking the LEXFIT prototype you designed and rebuilding it **1:1 as a production web app** — same screens, same feel, but powered by **real content and real users** instead of hardcoded mock data. It defines the stack, the database architecture, and an **8-phase build roadmap** you can hand to Claude Code one phase at a time. Target: **0 → MVP in two months**, starting September.

TARGET

Responsive web app

STACK

Next.js + Firebase

TIMELINE

~8 weeks to MVP

00 — HOW TO USE THIS DOCUMENT

The plan in one breath

You are non-technical and Claude Code will write the code. So this document is built to be **operated, not just read**. Each phase has a goal, a "definition of done" checklist, and a ready-to-paste kickoff prompt for Claude Code. You work top to bottom: finish a phase, confirm its checklist, move to the next. Nothing in a later phase depends on you understanding the code in an earlier one.

THE GOLDEN RULE

Build **back-to-front**: data first, then the screens that read it. We model the database before porting a single screen, so when the beautiful UI arrives in Phase 4 it plugs into real content instead of mock arrays. This is exactly the "database architecture first" order you asked for.

01 — WHAT WE'RE BUILDING

The product, as it exists in the prototype

The prototype is a Hungarian, women-first fitness app built around an empathetic, anti-"number-war" philosophy. It is not a generic workout library — it is a **guided journey**. Five surfaces make up the app shell:

SURFACE	WHAT IT IS	MVP?
Foundation home	The flagship 8-week / 40-workout program in 4 phases (Alap → Építés → Elmélyítés → Kifejezés). Fixed 30-min, equipment-free sessions, 5/week. The product's primary CTA.	Core
Videótár library	Filterable video library — phase, body-part, duration, difficulty, format, type. Includes bonus content (has-kihívás, reggeli flow, tartás, mobility).	Core
Haladásom progress	IRL-progress page centered on private progress photos at 3 milestones (Hét 1 / 5 / 8), non-scale wins, and the user's onboarding "why."	Core
Workout player player	The full-screen session view: video + block-by-block exercise breakdown + completion that updates progress & streak.	Core
Szavazz Magadra szm	Weekly community vote → 5-day vertical video drop with reactions/comments. A separate content track.	Phase 2 product

Plus the supporting flows: **onboarding** (collects height, weight, goal/"why", baseline), **sales / pricing pages**, and account/subscription management.

SCOPE DECISION — SZAVAZZ MAGADRA

You said "no voting yet" and MVP = the designs minus the most complex features. So for MVP we ship the four core surfaces and treat the **full Szavazz Magadra voting/community engine as the first post-launch project**. The data model below still leaves a clean seam for it, so adding it later is additive, not a rewrite.

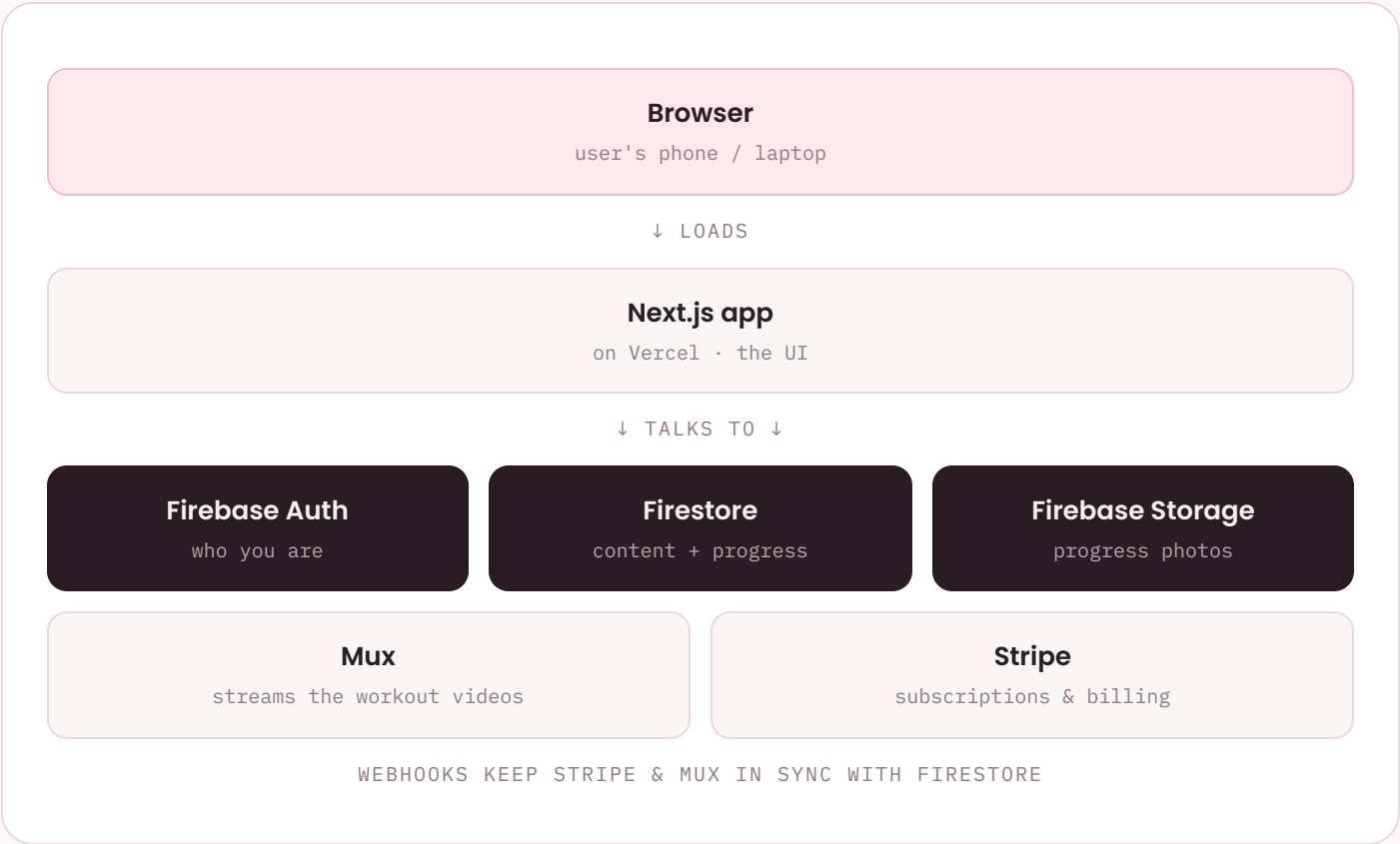
02 — THE STACK

What runs where

Every piece below is a service you already have an account for. Nothing exotic, nothing that locks you in beyond what you've already chosen.

FRAMEWORK	Next.js (App Router, React + TypeScript) — the closest 1:1 to your existing React prototype, so screens port with minimal rewriting.
HOSTING	Vercel — git-push deploys, preview URLs for every change, zero server management.
DATABASE	Firebase — Cloud Firestore (NoSQL document DB) for users, content metadata, progress, subscriptions.
AUTH	Firebase Authentication — Google, Apple & Facebook social login.
FILE STORAGE	Firebase Storage — progress photos, thumbnails, avatars (private, per-user rules).
VIDEO	Mux — upload, encode, adaptive streaming, signed playback for paid gating, thumbnails & analytics.
PAYMENTS	Stripe — recurring monthly/yearly subscription, Customer Portal, webhooks → Firestore.
SERVER LOGIC	Next.js API routes / Firebase Cloud Functions — webhooks, Mux signing, admin actions.
LANGUAGE	Hungarian only — but text held in one strings file so multi-language is a later switch, not a rebuild.

How it fits together



Read it top-down: the browser loads the Next.js app from Vercel; the app authenticates with Firebase, reads content & the user's progress from Firestore, streams video from Mux, and checks subscription status that Stripe keeps current via webhooks.

03 – DATABASE ARCHITECTURE

The Firestore data model

This is the heart of the rebuild — and it comes straight from your prototype's data files (`prog-data.jsx` , `lexfit-data.jsx` , `dash-data.jsx` , `szm-data.jsx`). Firestore organizes data into **collections** (think folders) of **documents** (think records). Below is the full set for MVP.

Content collections authored by you

COLLECTION	KEY FIELDS	FROM PROTOTYPE
<code>programs</code>	title, slug, weeks, totalWorkouts, perWeek, level, synopsis, facts[], phases[]	<code>PROG_META</code> , <code>PROG_PHASES</code>

COLLECTION	KEY FIELDS	FROM PROTOTYPE
<code>workouts</code>	code (F001...), title, theme, phase, week, day, mins, level, format, types[], muxAssetId, muxPlaybackId, thumb, blocks[{name,mins,items[]}], retest	<code>PROG_WEEKS</code> , <code>LX_VIDEOS</code> , <code>LX_TODAY_PLAN</code>
<code>bonusContent</code>	code (B/R/T/N/M...), title, series, theme, mins, level, format, muxPlaybackId	<code>LX_VIDEOS</code> (phase:null)
<code>filters</code>	phase, theme, dur, level, format, type – label + options	<code>LX_FILTERS</code>

Note: `workouts` and `bonusContent` can be one collection with a `kind` field — we'll decide in Phase 1. Each video document points at a Mux asset rather than holding the video itself.

User collections written by the app

COLLECTION	KEY FIELDS	PURPOSE
<code>users/{uid}</code>	displayName, email, photoURL, provider, createdAt, locale	Account profile (1 doc per user, keyed by Firebase Auth UID).
<code>users/{uid}/onboarding</code>	height, weight, goal, why (free text), experience, completedAt	The onboarding answers & baseline. Feeds Haladásom.
<code>users/{uid}/progress</code>	programId, joinedAt, currentIndex, doneCount, streak, completed[{code,at}], resume{code:seconds}	Program state — the live version of <code>PROG_DONE_COUNT</code> / <code>streak</code> / <code>LX_RESUME</code> .
<code>users/{uid}/photos</code>	milestone (1/5/8), storagePath, takenAt, note	Progress photos (private, in Firebase Storage). Body-image sensitive — locked-down rules.
<code>users/{uid}/myList</code>	codes[]	Saved videos — the "+" button. Replaces <code>localStorage lx_mylist</code> .
<code>users/{uid}/subscription</code>	status, plan, stripeCustomerId, currentPeriodEnd	Mirrors Stripe via webhook. The single source of truth for "is this user paid?"

DESIGN ↔ DATA MAPPING

Notice every mock constant in your prototype has a home here. `PROG_CURRENT_INDEX` → `progress.currentIndex` ; `DASH_WHY` → `onboarding.why` ; `dashMyList` → `myList` ; the `LX_RESUME` map → `progress.resume` . The migration is mechanical: **swap each hardcoded array/constant for a Firestore read.**

SECURITY RULES MATTER HERE

Firestore rules will enforce: content collections are **read-only to logged-in users** (and gated by subscription for the actual video playback IDs); each user can only read/write **their own** `users/{uid}/...` subtree. Progress photos are private to the owner. We write and test these rules in Phase 1 — before any real data goes in.

04 — REAL CONTENT, NOT MOCK DATA

Getting real videos & data in

You're starting fresh — no spreadsheet, no CMS yet. So part of this project is building the **pipeline that turns your real workouts into app content**. Three moving parts:

1 • Video → Mux

You upload each workout video to Mux. Mux encodes it and returns a `playbackId`. That ID goes into the workout's Firestore document.

2 • Metadata → Firestore

Title, phase, week, duration, format, the block-by-block exercise breakdown — entered once, stored as the `workouts` documents.

3 • Seed script

Your prototype already contains the full F001–F040 structure. We convert it into a one-time seed script so the program skeleton lands in Firestore instantly — you just attach videos.

For entering and maintaining that metadata you need an interface. That's the **admin dashboard** — and you mentioned you already have one from an e-learning project. See [Open Decisions](#); reusing it could save a week.

YOUR PROTOTYPE IS A HEAD START, NOT THROWAWAY

The 40-workout program, the 4-phase structure, the filter taxonomy, the "today's plan" block breakdown — all of it is real, considered content already written in Hungarian. We **seed it as the first real data**. The only thing genuinely missing is the actual video files behind each code.

05 — ACCOUNTS, ACCESS & MONEY

Who gets in, and what they can see

Authentication

Firebase Auth with Google, Apple and Facebook. A user signs in → gets a stable UID → that UID keys their entire data subtree. First-ever sign-in routes them into **onboarding** before the app.

Subscription gating

Stripe recurring plan (monthly & yearly). A Stripe webhook writes status into `users/{uid}/subscription`. The app reads that one field to decide: free preview vs. full access. Mux signed playback means non-subscribers literally can't stream paid videos.

The prototype already encodes the two states — "Csatlakozott · 4. hét" VS "Még nem csatlakozott" (joined vs. preview). That becomes the real free/paid boundary: previewers see Phase 0 / the program overview; subscribers get everything.

06 — THE BUILD ROADMAP

Eight phases, ~8 weeks

Each phase below is a self-contained chunk of work for Claude Code. Do them in order. The **grey prompt box** at the end of each is your kickoff message — paste it into Claude Code in VS Code to start that phase. Tell Claude Code to read this document first.

BEFORE PHASE 0 • ONE-TIME SETUP

In VS Code: open an empty folder, install the GitHub extension, and connect it to a new private repo. Have your Firebase, Vercel, Mux and Stripe logins handy — Claude Code will tell you exactly which keys to copy and where to paste them. You never have to write code; you copy keys and confirm checklists.

Phase 0 Week 1 • Foundations

Project skeleton & the design system

Stand up an empty-but-deployable Next.js app on Vercel, and port your LEXFIT design tokens so everything built afterwards already looks like the brand.

- Next.js + TypeScript project, pushed to GitHub, auto-deploying to Vercel.
- Firebase project connected (config + SDK wired, nothing stored yet).
- Port `lexfit-tokens.css` → global styles + Poppins / IBM Plex Mono fonts.
- The shared UI atoms (chips, cards, icons, cover art) rebuilt as reusable components.

DONE WHEN

- A live Vercel URL shows a branded "LEXFIT" placeholder page in the right fonts & colors.
- The repo is on GitHub and every push redeploys automatically.

● PASTE INTO CLAUDE CODE

```
# Phase 0 - LEXFIT foundations
Read "LEXFIT Build Plan.html" in the project for full context.
Scaffold a Next.js (App Router, TypeScript) app and deploy it to
Vercel via GitHub. Wire up the Firebase SDK using the project I
created (I'll paste the config). Port the design tokens from my
prototype's lexfit-tokens.css into global CSS, load the Poppins and
IBM Plex Mono fonts, and rebuild the shared UI atoms (chip, card,
icon, cover) as React components. End with a branded placeholder
page live on Vercel. Tell me exactly which keys to copy and where.
```


Database architecture & content seed

The core of "database first." Model every collection from Section 03, lock down security rules, and seed the real Foundation program from your prototype data.

- Define Firestore collections: `programs`, `workouts`, `bonusContent`, `filters`, and the `users/{uid}/...` subtree.
- Write & test Firestore security rules (content read-only; user data owner-only).
- Convert `prog-data.jsx` + `lexfit-data.jsx` into a seed script → real documents (videos attached later).

DONE WHEN

- Firestore shows the full 8-week / 40-workout program + filter taxonomy as real documents.
- Security rules pass a test: a logged-out request is denied; a user can't read another user's data.

● PASTE INTO CLAUDE CODE

```
# Phase 1 – database architecture
Using the data model in Section 03 of "LEXFIT Build Plan.html",
create the Firestore collections and TypeScript types for programs,
workouts, bonusContent, filters, and the users/{uid} subtree
(onboarding, progress, photos, myList, subscription). Write Firestore
security rules: content is read-only to authed users, each user can
only touch their own subtree, photos are owner-private. Then build a
seed script that converts my prototype's prog-data.jsx and
lexfit-data.jsx into real Firestore documents (leave Mux IDs blank
for now). Show me the data in the Firebase console when done.
```

Auth & onboarding

Real accounts, and the onboarding flow that creates each user's baseline.

- Firebase Auth with Google / Apple / Facebook sign-in.
- First sign-in → create `users/{uid}` → route into onboarding.
- Port the onboarding screens 1:1; save height, weight, goal & "why" to Firestore.
- Protected routes: unauthenticated visitors see sales/login, not the app.

DONE WHEN

- You can sign in with Google, complete onboarding, and see your answers saved in Firestore.
- Signing out and back in restores your profile.

PASTE INTO CLAUDE CODE

```
# Phase 2 – auth & onboarding
Add Firebase Authentication with Google, Apple and Facebook social
login. On first sign-in, create the users/{uid} document and route
the user into onboarding. Port my prototype's onboarding screens 1:1
(LEXFIT Onboarding.html) and save the answers – height, weight, goal,
free-text "why", experience – to users/{uid}/onboarding. Add route
protection so only authenticated, onboarded users reach the app.
```

Mux video pipeline & the player

Make video real: upload to Mux, store playback IDs, stream into the workout player with signed (gated) playback.

- Mux integration: upload flow, store `muxPlaybackId` on each workout doc.
- Signed playback URLs via a server route — only valid for subscribed users.
- Port the workout player 1:1: Mux player + block-by-block breakdown + resume position.
- Completion writes to `progress` (done count, streak, resume map).

DONE WHEN

- A real uploaded video plays in the branded player; finishing it updates your streak in Firestore.
- Refreshing resumes where you left off.

PASTE INTO CLAUDE CODE

```
# Phase 3 – Mux video & player
Integrate Mux. Build an upload path that stores each workout's
muxPlaybackId on its Firestore document, and a server route that
issues signed playback URLs only for users with an active
subscription. Port my workout player 1:1 (app/screen-player.jsx) using
the Mux player, with the block-by-block exercise breakdown beside it.
Persist resume position to users/{uid}/progress.resume, and on
completion update doneCount, streak and the completed[] log.
```

App shell + Foundation + Videótár (the 1:1 port)

The big one. The full app shell and the two core content surfaces, reading entirely from real Firestore data.

- App shell: sidebar nav + routing (Foundation / Videótár / Haladásom).
- Foundation home: phases, weeks, today's session, join/preview states — driven by real `progress`.
- Videótár: the full filter model (phase, theme, duration, level, format, type) over real workouts.
- "My list" (+ button) backed by Firestore, not localStorage.

DONE WHEN

- Foundation & Videótár look identical to the prototype but every card, filter and progress marker is live data.
- Free vs. subscribed users see the correct preview/full experience.

● PASTE INTO CLAUDE CODE

```
# Phase 4 — app shell, Foundation & Videótár
Port the app shell (app/lexfit-app-shell.jsx) with sidebar nav and
routing. Then port the Foundation home (app/screen-challenges.jsx +
program/*) and the Videótár (app/screen-library.jsx) 1:1, but read all
content and the user's program state from Firestore instead of the
mock arrays. Implement the full filter model from LX_FILTERS over real
workout documents, and back "my list" with users/{uid}/myList. Respect
the joined/preview (free vs subscribed) states throughout.
```

Haladásom — progress & photos

The emotional centerpiece: private progress photos at the three program milestones, non-scale wins, and the user's "why."

- Photo upload to Firebase Storage at Hét 1 / 5 / 8 milestones (private, owner-only).
- Before → now comparison; locked future milestones with unlock copy.
- Non-scale wins pulled from real **progress** (streak, completed, retest).
- "A miértem" recalls the onboarding free text.

DONE WHEN

- You can upload a milestone photo; it's private, persists, and shows in the comparison.
- Wins & "why" reflect your real account, not demo values.

PASTE INTO CLAUDE CODE

```
# Phase 5 — Haladásom  
Port the Haladásom progress screen (app/screen-progress.jsx,  
docs/haladasom-research.md) 1:1. Add private progress-photo upload to  
Firebase Storage at the Hét 1 / 5 / 8 milestones, owner-only via  
Storage rules, with the before-now comparison and locked future  
milestones. Populate the non-scale wins from users/{uid}/progress and  
recall the onboarding "why". Keep the privacy/self-compassion framing  
from the research doc exactly.
```

Stripe subscriptions & the sales pages

Turn on the business: recurring billing, the paywall, and the marketing front door.

- Stripe monthly & yearly plans; Checkout + Customer Portal.
- Webhook → `users/{uid}/subscription` keeps access status current.
- Port the sales / pricing pages 1:1; wire the CTA to Checkout.
- Gating verified end-to-end against Mux signed playback.

DONE WHEN

- A test purchase flips your account to full access; cancelling reverts it.
- Non-subscribers cannot stream paid videos even via direct link.

PASTE INTO CLAUDE CODE

```
# Phase 6 – Stripe & sales pages
Add Stripe recurring subscriptions (monthly + yearly) with Checkout
and the Customer Portal. Build the webhook that mirrors subscription
status into users/{uid}/subscription, and make that field the single
gate for full access (it must align with the Mux signed-playback check
from Phase 3). Port my sales/pricing pages 1:1 and wire their CTAs to
Checkout. Walk me through Stripe test mode to verify upgrade and
cancel.
```

Admin dashboard (content management)

So you can run the app without touching code: add workouts, upload videos, edit programs.

- Admin-only area (locked to your account) to create/edit workouts & bonus content.
- Mux upload + metadata entry in one form; publish toggles content live.
- **Decision point:** adapt your existing e-learning admin, or build fresh — see Open Decisions.

DONE WHEN

- You can add a brand-new workout — video + metadata — entirely through the dashboard, and it appears in the app.

● PASTE INTO CLAUDE CODE

```
# Phase 7 — admin dashboard  
Build an admin-only area (restricted to my account by a custom claim  
or allowlist) for managing content: create/edit workouts and bonus  
content, upload videos to Mux from the form, fill metadata, and  
publish/unpublish. If I share my existing e-learning admin project,  
first assess whether to adapt its UI/components or build fresh, then  
recommend and proceed.
```

QA, polish & go-live

Real content loaded, edges smoothed, ship it.

- Load real videos & final copy; remove every remaining demo value.
- Mobile responsiveness pass across all surfaces.
- Error/empty/loading states; analytics; basic SEO & the custom domain.
- Pre-launch run-through: sign up → subscribe → complete a workout → see progress.

DONE WHEN

- A new real user can go from landing page to completing their first workout with zero mock data anywhere.

Five things to settle before/early in the build

#	DECISION	WHY IT MATTERS / WHAT I NEED
1	The e-learning admin	You have an existing admin dashboard. Share that repo and I'll judge whether adapting it (could save ~a week in Phase 7) beats building fresh. The deciding factor is its stack — if it's already React/Next, reuse is easy.
2	Free vs. paid boundary	What exactly can a non-subscriber see? Suggested: the program overview + Phase 0 (Alap) as a taste, everything else gated. Confirm or adjust.
3	Pricing	Actual monthly & yearly prices for the Stripe plans (can be placeholders for now, but Phase 6 needs numbers).
4	Video readiness	How many of the 40 workouts are actually filmed? This sets whether we launch the full program or a first phase. The app architecture doesn't change either way.
5	Szavazz Magadra timing	Confirmed out of MVP. Decide if even a static (non-voting) content view of it should appear at launch, or be fully deferred.

Immediate next steps

1. Confirm this plan & the stack (or tell me what to change).
2. Answer the five decisions above — especially share the e-learning admin repo.
3. I produce, if you want, the **VS Code project scaffold + a** `CLAUDE.md` so Claude Code opens the project already knowing all of this context.
4. Begin Phase 0.

WHY THIS IS VERY ACHIEVABLE IN TWO MONTHS

You are not designing as you build — the hardest, most ambiguous work is **already done** in your prototype. Every screen, interaction, color, and even the content taxonomy exists. This rebuild is largely a disciplined port plus wiring real services you've already signed up for. The phased order keeps each week shippable and testable.

